

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
ВВЕДЕНИЕ.....	5
1 Обзор предметной области	7
1.1 Роль программной документации в open-source репозиториях.....	7
1.2 README-файл как основной элемент документационного сопровождения проекта	10
1.3 Проблемы качества и сопровождения документации в открытых репозиториях.....	13
1.4 Автоматизация генерации программной документации.....	14
1.5 Использование больших языковых моделей для анализа кода и генерации документации.....	16
1.6 Мультиагентный подход к задачам программной инженерии.....	17
2 Анализ существующих решений и подходов.....	20
2.1 Критерии сравнения систем генерации и сопровождения документации.....	20
2.2 Инструменты и подходы к автоматической генерации README- файлов.....	21
2.3 Репозиторно-ориентированные системы генерации кодовой документации.....	24
2.4 Мультиагентные системы в задачах программной инженерии.....	25
3 Проектирование и реализация мультиагентной системы генерации документации.....	27
3.1 Назначение и требования к системе	27
3.2 Общая архитектура программного комплекса	29

3.3	Архитектура пайплайна генерации README	32
3.4	Агенты пайплайна генерации README и их функции	35
4	Экспериментальное исследование и оценка результатов	43
4.1	Цель и методика экспериментального исследования	43
4.2	Экспериментальный набор репозитория	45
4.3	Критерии оценки качества README-файлов	46
4.4	Анализ результатов экспериментального исследования	48
	ЗАКЛЮЧЕНИЕ	50
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	51
	Приложение А. Программная реализация структуры ReadmeContext	55
	Приложение Б. Программная реализация структуры ReadmeState	56
	Приложение В. Экспериментальный набор репозитория	58
	Приложение Г. Prompt-шаблон для оценки качества README	59

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API – Application Programming Interface, программный интерфейс приложения
CI/CD – Continuous Integration / Continuous Delivery, непрерывная интеграция и доставка программного обеспечения

CLI – Command-Line Interface, интерфейс командной строки

FLOSS – Free/Libre and Open-Source Software, свободное и открытое программное обеспечение

G-Eval – метод автоматической оценки сгенерированного текста с использованием большой языковой модели

JSON – JavaScript Object Notation, текстовый формат обмена данными

LLM – Large Language Model, большая языковая модель

LLM-as-a-judge – подход к оценке результатов, при котором языковая модель используется в качестве оценщика

Markdown – легковесный язык разметки текстовых документов

NLG – Natural Language Generation, генерация естественного языка

OSA – Open-Source Advisor, программная система для автоматизированного анализа и улучшения open-source репозиторий

PDF – Portable Document Format, формат электронных документов

PyPI – Python Package Index, репозиторий пакетов Python

RAG – Retrieval-Augmented Generation, генерация с дополнением извлечённым контекстом

README – основной файл репозитория, содержащий описание проекта, инструкции по установке, запуску и использованию

URL – Uniform Resource Locator, унифицированный указатель ресурса

MAC – мультиагентная система.

ВВЕДЕНИЕ

Репозитории с открытым исходным кодом являются важной частью современной разработки программного обеспечения, поскольку обеспечивают распространение программных решений, совместную работу разработчиков и повторное использование результатов. При этом качество open-source проекта определяется не только корректностью исходного кода, но и полнотой сопроводительной документации. Согласно рекомендациям GitHub, README-файл используется для передачи ключевой информации о проекте, а вместе с лицензией, правилами участия в разработке и Code of Conduct помогает зафиксировать ожидания от проекта и организовать взаимодействие с участниками сообщества [1].

Несмотря на значимость документации, её создание и поддержание остаются трудоёмкими задачами. Разработчики тратят значительную долю времени на понимание программ, а качественная документация снижает эти затраты; при этом сопровождение документации требует времени, ресурсов и трудозатрат, поэтому не все проекты способны уделять ей достаточное внимание [2]. Следовательно, проблема неполной, устаревшей или несогласованной документации остаётся актуальной для open-source проектов.

Современные большие языковые модели позволяют автоматизировать генерацию текстовых артефактов и анализ программного кода, однако их прямое применение не всегда обеспечивает достаточную полноту и фактическую корректность результата. LLM-based решения для генерации документации могут пропускать существенную информацию, давать недостаточный контекст и формировать фактически некорректные ссылки на несуществующие компоненты, особенно в крупных репозиториях [3].

Перспективным способом решения данной проблемы является использование мультиагентного подхода. В такой архитектуре отдельные агенты могут выполнять специализированные функции: анализировать структуру репозитория, извлекать релевантный контекст, формировать

документацию, проверять её качество и вносить улучшения на основе обратной связи [3]. Подобная декомпозиция позволяет рассматривать генерацию документации не как однократный вызов языковой модели, а как итеративный процесс последовательного уточнения результата.

Целью настоящей работы является качества генерируемой программной документации для репозитория с открытым исходным кодом за счет применения мультиагентного подхода и механизма итеративного улучшения.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и существующих решений в области автоматизации генерации программной документации.
2. Исследовать современные подходы к применению больших языковых моделей для анализа программного кода и генерации документации.
3. Разработать архитектуру мультиагентной системы, обеспечивающей декомпозицию задач анализа, генерации, проверки и улучшения документации.
4. Реализовать модуль генерации README-файлов с учётом лучших практик open-source разработки.
5. Разработать механизм итеративного улучшения документации на основе обратной связи агентов.
6. Провести экспериментальное исследование эффективности разработанной системы.

Научно-техническая новизна работы заключается в разработке архитектуры мультиагентной системы, в которой генерация программной документации дополняется этапами автоматической оценки и итеративного улучшения.

Практическая значимость работы состоит в возможности применения разработанной системы в составе проекта OSA [4] для автоматизации подготовки и сопровождения open-source репозитория.

1 Обзор предметной области

1.1 Роль программной документации в open-source репозиториях

Программная документация является одним из ключевых элементов open-source репозитория, поскольку обеспечивает связь между исходным кодом проекта и его потенциальными пользователями, разработчиками и участниками сообщества. В отличие от закрытых программных решений, открытые репозитории предполагают возможность внешнего изучения, запуска, повторного использования и модификации кода. Следовательно, для таких проектов особенно важны материалы, которые позволяют понять назначение программного продукта, порядок его установки, основные сценарии применения и правила участия в развитии проекта.

Наиболее заметным элементом документационного сопровождения является README – предварительный источник информации о GitHub-проекте, который помогает разработчикам понять назначение репозитория для его повторного использования или расширения. Обычно именно он выступает первой точкой взаимодействия пользователя с репозиторием и формирует первичное представление о проекте.

Значимость README подтверждается эмпирическими данными. Анализ 1950 публичных GitHub-репозиториях, охватывающих десять языков программирования, показал статистическую связь между структурой README-файлов и популярностью проектов. Популярность репозитория оценивалась не только по количеству stars, но и с учётом forks, watchers и pull requests. Установлено, что популярные репозитории чаще содержат структурно организованные README-файлы, включающие списки, изображения, внешние ссылки и ссылки на другие GitHub-ресурсы. Дополнительно было выявлено, что наличие разделов с contribution guidelines и references связано с более высокой популярностью проекта [5]. На рисунке 1 показано распределение структурных признаков README-файлов

относительно ранга популярности репозитория. Увеличение числа выраженных пиков при росте popularity rank указывает на то, что некоторые признаки README чаще встречаются в более популярных проектах. Наиболее заметная связь наблюдается для внешних ссылок, ссылок на GitHub-ресурсы, списков и изображений.

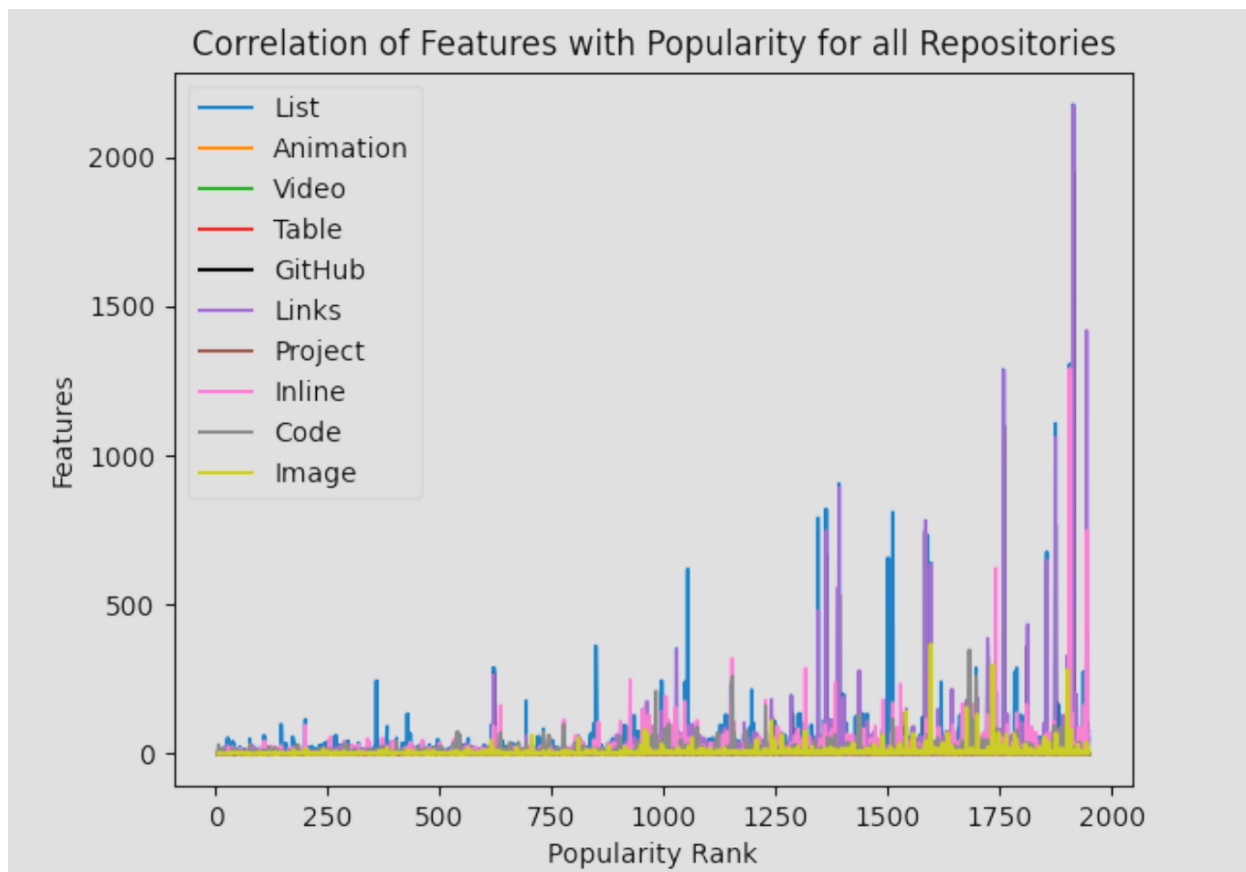


Рисунок 1 – Связь структурных признаков README-файлов с популярностью GitHub-репозитория [5]

При этом документационное сопровождение open-source проекта не ограничивается только README. Для устойчивого развития репозитория также значимы инструкции по участию в разработке, описание правил поведения в сообществе, сведения о лицензировании, примеры использования и ссылки на расширенную документацию. Совокупность этих материалов формирует внешний интерфейс проекта для пользователей и разработчиков, не знакомых с его внутренней реализацией.

Значимость документации связана с тем, что работа с программным проектом начинается не только с чтения исходного кода, но и с его

интерпретации. Недостаток поясняющих материалов увеличивает время, необходимое для понимания назначения модулей, зависимостей, сценариев запуска и ограничений системы [2]. Следовательно, документация может рассматриваться как средство уменьшения когнитивной нагрузки при изучении репозитория и как фактор, влияющий на скорость включения новых участников в разработку.

Роль документации проявляется не только на уровне отдельного GitHub-репозитория, но и на уровне программных экосистем. В исследовании, посвященном npm-экосистеме, изучались характеристики пакетов, которые часто выбираются разработчиками в качестве зависимостей. В рамках качественной части исследования были опрошены 118 JavaScript-разработчиков. Согласно результатам опроса, документация оказалась наиболее важным фактором при выборе npm-пакета: медианная оценка составила 5,0 из 5, а среднее значение — 4,64 [6]. Из данных таблицы 1 следует, что разработчики при выборе программного пакета ориентируются не только на показатели популярности, но и на наличие понятной и достаточно полной документации.

Таблица 1 – Оценка факторов, влияющих на выбор npm-пакета [6]

Фактор	Медиана	Среднее значение
Documentation	5,0	4,64
Downloads	4,5	4,30
Stars	4,0	3,97
Vulnerabilities	4,0	3,70
Release	4,0	3,48
Commit frequency	3,0	3,33
Closed issue	3,0	3,29
License	3,0	3,26
Usage	3,0	3,19
Test Code	3,0	3,14

Помимо поддержки понимания кода, документация влияет на воспроизводимость работы программного решения и сопровождение проекта во времени. Для open-source репозитория важно не только предоставить исходный код, но и описать условия, при которых он может быть установлен, запущен и проверен. Отсутствие инструкций по установке, сведений о зависимостях или примеров запуска может сделать проект формально открытым, но практически труднодоступным для внешнего пользователя. Кроме того, по мере развития кодовой базы изменяются зависимости, интерфейсы, команды запуска, структура директорий и сценарии использования. Если документация не обновляется одновременно с кодом, она теряет согласованность с фактическим состоянием репозитория. Создание и поддержание документации является трудоёмким и подверженным ошибкам процессом, а документация в open-source проектах может отставать от изменений кода [3].

Таким образом, программная документация в open-source репозиториях выполняет несколько взаимосвязанных функций: обеспечивает первичное понимание проекта, поддерживает воспроизводимость его установки и запуска, задаёт правила участия в разработке и способствует сопровождению кодовой базы. Эмпирические исследования показывают, что содержание README-файлов и качество документации связаны с популярностью репозитория и выбором open-source компонентов разработчиками. Данное обстоятельство определяет необходимость дальнейшего рассмотрения README как центрального элемента документационного сопровождения open-source проекта.

1.2 README-файл как основной элемент документационного сопровождения проекта

README-файл занимает центральное место в документационном сопровождении open-source репозитория, поскольку выполняет функцию

первичной точки входа к проекту. Как правило, он размещается в корневой директории и автоматически отображается на странице репозитория, поэтому пользователь знакомится с ним раньше, чем с исходным кодом, конфигурационными файлами или дополнительной документацией. Содержательная специфика README заключается в объединении нескольких типов информации. В одном документе могут присутствовать краткое описание проекта, сведения о функциональных возможностях, требования к окружению, команды установки, примеры запуска, ссылки на расширенную документацию, сведения о лицензии и правила участия в разработке. Поэтому README нельзя рассматривать только как аннотацию к репозиторию. По своей функции он ближе к навигационному документу, связывающему исходный код, пользовательские сценарии и сопутствующие материалы проекта.

Структура README-файлов неоднократно становилась объектом эмпирических исследований. В одном из таких исследований были проанализированы 4226 секций из 393 README-файлов GitHub-репозитория; на основе разметки были выделены категории содержания, отражающие основные типы информации, встречающиеся в таких документах [7]. Классификация, представленная в таблице 2, показывает, что README обычно отвечает не на один, а на несколько разных вопросов пользователя: что представляет собой проект, зачем он нужен, как его установить, как использовать, где получить дополнительную информацию и каким образом можно принять участие в разработке.

Таблица 2 – Основные категории содержания README-файлов [7]

Категория	Содержание категории	Назначение в README
What	Описание функциональности проекта	Позволяет понять, что реализует проект
Why	Назначение и мотивация проекта	Объясняет, зачем создано программное решение

How	Инструкции по установке и использованию	Обеспечивает возможность практического запуска
When	Сведения о состоянии, версиях и релизах	Показывает актуальность и стадию развития проекта
Who	Информация об авторах и участниках	Помогает определить сопровождающих проекта
References	Ссылки на API, документацию и дополнительные материалы	Расширяет контекст использования проекта
Contribution	Правила участия в разработке	Определяет порядок внесения изменений

Приведённые категории важны для задачи автоматической генерации README. Система, формирующая такой документ, должна не только генерировать связный текст, но и определять, какие содержательные блоки необходимы конкретному репозиторию.

Отдельное значение имеет формат представления. В большинстве современных репозиториях README оформляется с использованием Markdown, поскольку этот формат поддерживает заголовки, списки, ссылки, изображения, таблицы и блоки кода. GitHub также использует заголовки Markdown для формирования навигационной структуры документа [8]. Поэтому качество README определяется не только полнотой содержания, но и тем, насколько удобно это содержание организовано для чтения.

В исследовании, посвящённом появлению README и CONTRIBUTING в проектах свободного и открытого программного обеспечения (Free/Libre and Open Source Software, FLOSS), показано, что эти документы вводятся на разных этапах жизненного цикла: README обычно появляется в самом начале проекта, медианное время появления совпадает с днем первого коммита [9]. При этом раннее наличие README не означает его достаточности. Первые версии документа часто бывают очень короткими и в

основном сосредоточены на технических инструкциях по установке и использованию, а не на полноценном описании проекта или развитии сообщества [9]. Следовательно, README может присутствовать в репозитории формально, но не выполнять свою функцию в полном объёме.

Для настоящей работы это наблюдение имеет принципиальное значение. Разрабатываемая система должна быть ориентирована не только на факт наличия README, но и на его содержательную полноту, структурную организованность и пригодность для дальнейшего сопровождения. При генерации такого файла необходимо учитывать назначение проекта, структуру кода, технологический стек, зависимости, команды установки и запуска, примеры использования и ссылки на дополнительные материалы.

1.3 Проблемы качества и сопровождения документации в открытых репозиториях

Наличие документации в open-source репозитории не гарантирует её достаточности для практического использования проекта. README-файл может существовать формально, но не содержать сведений, необходимых для установки, запуска, понимания структуры проекта или участия в разработке. Такая ситуация особенно характерна для проектов, в которых документация создаётся на ранних этапах и затем не развивается синхронно с кодовой базой.

Проблема качества документации не сводится только к фактической точности. В исследовании качества программной документации предложен подход, согласно которому оценка должна учитывать не только ассурасу и completeness, но и более широкий набор характеристик: понятность, читаемость, структуру, связность, краткость, согласованность терминологии и ясность изложения [10]. Такой подход важен для open-source проектов, поскольку пользователь взаимодействует с документацией как с самостоятельным информационным продуктом, а не только как с приложением к исходному коду.

Дополнительной проблемой является сопровождение документации во времени. По мере развития проекта изменяются зависимости, интерфейсы, команды запуска, конфигурационные параметры и сценарии использования. Если документация не обновляется вместе с кодом, она постепенно теряет актуальность. Поддержание документации требует значительных трудозатрат, а её несогласованность с кодом остаётся одной из типичных проблем программных проектов [2, 3].

1.4 Автоматизация генерации программной документации

Автоматизация генерации программной документации рассматривается как один из способов снизить трудозатраты на сопровождение open-source репозиторий. В отличие от ручного написания README, инструкций по установке или описаний программных компонентов, автоматизированный подход позволяет быстрее получать первичный вариант документации и затем дорабатывать его с учётом структуры проекта. Это особенно важно для репозиторий, в которых кодовая база развивается быстрее, чем сопроводительные материалы.

При этом автоматизация документации не является однородной задачей, поскольку разные виды документационного сопровождения требуют различного уровня анализа исходного кода. Формирование комментария к отдельной функции или методу связано преимущественно с интерпретацией локального фрагмента программы. Напротив, описание проекта как целостного программного продукта предполагает определение его общей цели, основных компонентов, технологического стека, зависимостей, точек входа и пользовательских сценариев. В исследованиях, посвящённых автоматическому созданию README, подчёркивается, что такой документ должен формировать абстрактное описание проекта на основе большого объёма исходного кода [11]. Следовательно, передача всего содержимого репозитория в языковую модель не является устойчивым решением: объём

кодовой базы может превышать ограничения контекстного окна, а значительная часть файлов не будет иметь прямого отношения к содержанию итоговой документации.

Один из возможных способов решения данной проблемы состоит в предварительном выделении репрезентативной части кода. Такой подход предполагает поиск файла или группы файлов, которые наиболее полно отражают назначение и устройство репозитория. Экспериментальные результаты показывают, что использование репрезентативного кода в качестве контекста позволяет повысить полезность и фактическую корректность автоматически сгенерированных README по сравнению с вариантом, в котором контекст формируется на основе случайно выбранного файла [11].

Однако для более сложных репозиториях одного репрезентативного файла может быть недостаточно. Проект может включать несколько точек входа, набор взаимосвязанных модулей, конфигурационные файлы, тесты, примеры запуска и существующие фрагменты документации. В таких случаях автоматизация требует не только генерации текста, но и предварительного понимания структуры репозитория. Для этого могут использоваться методы структурного анализа кодовой базы. В современных подходах к автоматизированному пониманию репозитория применяются иерархические деревья кода, графы зависимостей модулей, графы вызовов функций и механизмы выделения ключевых компонентов. Такие методы позволяют не передавать языковой модели весь репозиторий целиком, а отбирать информацию, наиболее значимую для текущей задачи [12]. Для генерации документации это означает необходимость предварительно определить, какие части проекта действительно описывают его назначение и способы использования.

Таким образом, автоматизация генерации программной документации не сводится к однократному вызову языковой модели. Она включает предварительный анализ репозитория, извлечение релевантного контекста, формирование входных данных для генерации и последующую проверку

результата. Такая логика подводит к необходимости более сложной архитектуры, в которой анализ, генерация и оценка документации рассматриваются как связанные этапы одного процесса.

1.5 Использование больших языковых моделей для анализа кода и генерации документации

Большие языковые модели стали одним из основных инструментов автоматизации задач, связанных с обработкой программного кода и технического текста. Их применение в программной инженерии обусловлено тем, что значительная часть инженерных артефактов может быть представлена в текстовой форме: исходный код, комментарии, сообщения об ошибках, инструкции по установке, описания API, README-файлы и пользовательские руководства. Благодаря этому задачи анализа кода и генерации документации могут рассматриваться как задачи преобразования и обобщения информации, в которых модель связывает программные конструкции с их естественно-языковым описанием.

LLM используются в широком спектре задач программной инженерии: генерации программного кода, автоматического дополнения уже написанных фрагментов, программного ремонта, тестирования, суммаризации кода, создания комментариев и сопровождения программных артефактов [13]. Для документационного сопровождения особенно важны способности моделей к обобщению и переформулированию. Такие модели могут извлекать смысл из фрагментов кода, выделять назначение функций и классов, преобразовывать технические детали в связный текст и адаптировать описание под заданный формат документа.

В случае README-файлов и сопутствующей документации роль языковой модели состоит не только в генерации текста. Модель должна связать между собой разнородные сведения: назначение проекта, структуру репозитория, зависимости, команды запуска, примеры использования и

ограничения. Такой процесс ближе к технической суммаризации, чем к обычной текстовой генерации. От качества входного контекста здесь зависит не только полнота результата, но и его фактическая корректность.

Однако возможности LLM не устраняют необходимость контроля результата. При генерации программной документации модели могут пропускать важные сведения, давать недостаточно подробное объяснение или формировать утверждения, не соответствующие реальному содержимому репозитория [3]. Одной из причин таких ошибок является зависимость модели от предоставленного контекста. Если во входные данные не попали ключевые файлы, точка входа, конфигурация запуска или сведения о зависимостях, модель может сформировать правдоподобный, но непроверенный текст. При этом увеличение объёма передаваемого контекста не всегда решает проблему: длинные входные данные могут превышать ограничения контекстного окна, содержать много нерелевантной информации и снижать способность модели выделять действительно важные элементы.

Поэтому использование LLM в генерации документации требует дополнительной инженерной обвязки. Необходимо заранее определить, какие сведения из репозитория должны быть переданы модели, каким образом они будут структурированы и как будет проверяться полученный результат. Без таких механизмов языковая модель остаётся инструментом генерации текста, но не полноценной системой документационного сопровождения.

1.6 Мультиагентный подход к задачам программной инженерии

Использование одной языковой модели для анализа репозитория и генерации документации имеет естественные ограничения. Такая модель может обрабатывать код, формировать текст и выполнять отдельные рассуждения, однако сложные задачи программной инженерии обычно требуют нескольких разных операций: поиска релевантных файлов, анализа зависимостей, интерпретации назначения компонентов, генерации результата

и его проверки. Объединение всех этих функций в одном запросе делает процесс трудноуправляемым и повышает риск неполного или фактически некорректного вывода.

Мультиагентный подход предлагает иную организацию работы. Вместо единого универсального компонента используется несколько специализированных агентов, каждый из которых отвечает за отдельный этап решения задачи. Один агент может анализировать структуру репозитория, другой — извлекать релевантный контекст, третий — формировать документацию, а отдельный проверяющий агент может оценивать полноту, согласованность и корректность результата. Такая декомпозиция соответствует общей логике программной инженерии, где сложный процесс разделяется на роли, этапы и промежуточные артефакты. Мультиагентные системы (МАС) позволяют решать более сложные задачи за счёт специализации агентов, совместной проверки решений и распределения ответственности между компонентами [14]. Важную роль при этом играет не только наличие нескольких агентов, но и способ их координации. Мультиагентная система должна определять, какие данные передаются между агентами, в каком порядке выполняются операции, как фиксируются промежуточные результаты и каким образом учитывается обратная связь.

Идея распределения ролей получила развитие в современных МАС для разработки программного обеспечения. В таких подходах агенты могут выполнять функции проектировщика, разработчика, проверяющего или координатора, а сам процесс строится как последовательность взаимосвязанных этапов [15, 16]. Для настоящей работы важен принцип повышения качества результата за счёт разбиения сложной задачи на подзадачи и проверки промежуточных результатов. В задачах генерации документации этот принцип проявляется особенно явно. Документация должна быть одновременно содержательной, структурированной, понятной и согласованной с кодом. Поэтому её создание целесообразно рассматривать не как однократную генерацию текста, а как процесс, включающий анализ,

генерацию, оценку и уточнение. Такой подход соответствует современным решениям, в которых специализированные агенты последовательно извлекают контекст, формируют документацию и проверяют её качество по нескольким критериям [3].

Отдельное значение имеет итеративность. Если проверяющий агент выявляет отсутствие важного раздела, недостаток контекста или несоответствие документации коду, результат должен быть возвращён на доработку. В этом случае обратная связь становится не внешним комментарием пользователя, а частью внутреннего механизма системы.

Таким образом, мультиагентный подход позволяет связать анализ репозитория, генерацию документации и проверку результата в единую управляемую процедуру. Для разрабатываемой системы это означает необходимость выделения специализированных компонентов, отвечающих за разные стадии обработки: анализ структуры проекта, извлечение контекста, генерацию README и сопутствующей документации, оценку качества и итеративное улучшение результата.

2 Анализ существующих решений и подходов

2.1 Критерии сравнения систем генерации и сопровождения документации

Анализ существующих решений в области автоматической генерации и сопровождения программной документации требует предварительного определения критериев сравнения. Рассматриваемые системы решают разные задачи: одни ориентированы на формирование README-файлов, другие — на генерацию документации к функциям, классам и модулям, третьи — на анализ структуры репозитория или организацию мультиагентного взаимодействия. Поэтому их сопоставление только по факту использования LLM не позволяет определить, насколько решение применимо к задаче настоящей работы.

В качестве первого критерия рассматривается тип генерируемой документации. Для данной работы наибольший интерес представляют решения, связанные с README-файлами и сопутствующими документами open-source репозитория. Вместе с тем в анализ включаются и системы генерации кодовой документации, поскольку они решают близкую задачу преобразования информации из исходного кода в естественно-языковое описание [2, 3].

Второй критерий связан с уровнем анализа репозитория. Система может работать с отдельным файлом, выбранным фрагментом кода, набором файлов или репозиторием как целостным объектом. Для генерации README особенно важен репозиторный уровень, поскольку итоговый документ должен отражать назначение проекта, его структуру, зависимости и пользовательские сценарии. Поэтому отдельно учитывается способ отбора контекста для LLM: использование репрезентативного кода, структурного анализа, графов зависимостей или других механизмов выделения значимой информации [11, 12].

Также важны наличие проверки качества результата и поддержка итеративного улучшения. Документация должна оцениваться не только с точки зрения языковой корректности, но и по полноте, понятности, структурированности, согласованности с кодом и фактической достоверности [10]. Если система способна выявлять недостатки сгенерированного текста и возвращать результат на доработку, она лучше соответствует задаче генерации не только первичного черновика, но и пригодной к сопровождению документации.

Отдельно рассматривается архитектурная организация решения. Системы могут строиться как линейный pipeline или как мультиагентная архитектура, в которой отдельные компоненты отвечают за анализ, генерацию, проверку и координацию. Для настоящей работы этот критерий является принципиальным, поскольку разрабатываемая система ориентирована на распределение функций между специализированными агентами [14].

Таким образом, дальнейший анализ существующих решений будет проводиться по следующим критериям: тип документации, уровень анализа репозитория, способ формирования контекста, использование LLM, наличие проверки качества, поддержка итеративного улучшения, архитектурная организация и практическая применимость. Такая схема позволит сопоставить различные подходы и определить ограничения, которые необходимо учесть при проектировании собственной системы.

2.2 Инструменты и подходы к автоматической генерации README-файлов

Автоматическая генерация README-файлов является отдельным направлением в области документационного сопровождения программных проектов. Такая задача требует получить обобщённое представление о проекте в целом: определить его назначение, основные возможности, способ установки, команды запуска и потенциальные сценарии использования.

Поэтому системы генерации README должны решать не только задачу текстовой генерации, но и задачу отбора релевантной информации из репозитория.

Одним из подходов к решению этой проблемы является использование LLM совместно с предварительным выделением репрезентативного фрагмента кода. В LARCH предлагается сначала определить файл, который наиболее полно отражает назначение репозитория, а затем использовать его как основу для prompt, передаваемого языковой модели [11]. Такой подход связан с ограничением контекстного окна LLM: вместо попытки передать модели весь репозиторий система сокращает входные данные до наиболее информативного фрагмента.

Архитектура LARCH включает несколько этапов. Сначала система получает репозиторий из рабочей области пользователя, затем выполняет выделение репрезентативного кода с использованием эвристических признаков и weak supervision. После этого на основе найденного фрагмента кода формируется prompt, который передается LLM для генерации README. В работе также предусмотрены интерфейсы для Visual Studio Code и CLI, что делает систему применимой не только как исследовательский прототип, но и как инструмент, который может быть встроен в рабочий процесс разработчика [11].

Экспериментальные результаты показывают, что выбор контекста существенно влияет на качество README. В сравнении с базовым подходом, использующим случайно выбранный файл, подход с representative code чаще приводил к полезным результатам: 65 % сгенерированных README были оценены как полезные против 40 % у базового подхода. Также повысилась фактическая корректность: для текстовых утверждений показатель составил 75 % против 55 %, а для фрагментов кода — 65 % против 30 % [11].

Однако данный подход имеет ограничения. Использование одного репрезентативного файла может быть достаточным для небольших или хорошо структурированных проектов, но не всегда отражает сложные

репозитории с несколькими точками входа, набором модулей и разнообразными пользовательскими сценариями.

Другим направлением являются подходы, связанные не с созданием полного README с нуля, а с извлечением и суммаризацией информации из уже существующего артефакта. К этому направлению относится Metagente — мультиагентный подход к суммаризации README.MD-файлов. Его задача состоит в генерации краткого About-описания репозитория на основе содержимого исходного файла. Используется несколько специализированных агентов. Extractor Agent удаляет нерелевантные и шумовые части README, оставляя фрагменты, связанные с описанием проекта. Summarizer Agent формирует краткое описание, Teacher Agent анализирует результат с учётом ground-truth About и ROUGE-L, а Prompt Creator Agent формирует улучшенный prompt для последующих итераций [17]. В отличие от однократной генерации, этот подход строится вокруг итеративного уточнения prompt и использования обратной связи.

По результатам экспериментов Metagente сравнивался с GitSum, LLaMA-2 и GPT-4o. Авторы указывают, что предложенный подход превосходит базовые подходы при генерации About-описаний для GitHub README.MD; прирост по сравнению с GitSum составляет от 27,63 % до 60,43 % [17].

Таким образом, существующие подходы к автоматизации README можно условно разделить на два типа. Первый тип ориентирован на генерацию README из исходного кода и требует отбора репрезентативного контекста из репозитория. Второй тип работает с уже существующим README и решает задачу его сокращения, структурирования или извлечения ключевого описания проекта. Оба направления полезны для настоящей работы, однако каждое из них покрывает только часть целевой задачи.

2.3 Репозиторно-ориентированные системы генерации кодовой документации

Помимо инструментов, ориентированных на README-файлы, существуют решения для генерации кодовой документации на уровне репозитория. Они не полностью совпадают с задачей настоящей работы, поскольку чаще описывают функции, классы и модули, а не формируют основной документационный файл проекта. Тем не менее такие системы важны для анализа, так как показывают способы извлечения контекста из кодовой базы и преобразования его в структурированное описание.

Одним из таких решений является RepoAgent — LLM-based фреймворк для генерации и сопровождения документации на уровне репозитория. Система использует глобальный контекст репозитория, извлекает метаинформацию о кодовых объектах и учитывает отношения между ними. На основе этих данных формируется документация, включающая описание функциональности, параметров, особенностей использования и примеры [2]. Дополнительно RepoAgent поддерживает обновление документации при изменениях кода с использованием Git-механизмов, что позволяет рассматривать документацию как сопровождаемый артефакт, связанный с жизненным циклом проекта [2].

Другим примером является DocAgent, где генерация документации организована как мультиагентный процесс. Система строит порядок обработки компонентов с учётом зависимостей, после чего специализированные агенты анализируют код, извлекают контекст, формируют документацию и проверяют её качество. Для оценки результата используются критерии полноты, полезности и фактической согласованности документации с кодом, что подчёркивает необходимость проверять не только языковую корректность, но и содержательную достоверность сгенерированного описания [3].

Таким образом, RepoAgent и DocAgent демонстрируют два значимых принципа для настоящей работы: использование репозиторного контекста и включение этапа проверки результата.

2.4 Мультиагентные системы в задачах программной инженерии

Мультиагентные системы на основе LLM активно применяются в задачах программной инженерии, поскольку разработка программного обеспечения естественным образом разделяется на несколько видов деятельности: анализ требований, проектирование, реализацию, тестирование, отладку и сопровождение.

Одним из известных примеров является MetaGPT. В этой системе процесс разработки строится на основе Standard Operating Procedures, то есть формализованных процедур, задающих роли, порядок действий и структуру промежуточных артефактов. В MetaGPT агенты выполняют роли Product Manager, Architect, Project Manager, Engineer и QA Engineer, а взаимодействие между ними организуется через структурированные документы, включая требования, проектные артефакты, интерфейсные спецификации и тестовые материалы [15].

Другой пример — ChatDev, где процесс разработки представлен как последовательность фаз design, coding и testing. Система использует chat chain, в рамках которой агенты с разными ролями взаимодействуют в многошаговом диалоге и последовательно решают подзадачи. Дополнительно применяется механизм communicative dehallucination: агенты могут запрашивать уточняющую информацию перед тем, как выдавать окончательный ответ, что направлено на снижение риска неполных или некорректных решений [16].

Для задач, связанных с реальными репозиториями, особое значение получает не только распределение ролей, но и способность системы исследовать кодовую базу. В RepoMaster эта задача решается через построение иерархического дерева кода, графа зависимостей модулей и графа вызовов

функций. Такая структура позволяет агенту не читать весь репозиторий целиком, а выделять ключевые компоненты и постепенно расширять контекст в процессе выполнения задачи [12]. Этот принцип близок к требованиям системы генерации документации, поскольку качественный README также требует отбора релевантной информации из проекта.

Таким образом, рассмотренные системы демонстрируют несколько принципов, значимых для настоящей работы: распределение ролей между агентами, структурированную коммуникацию, работу с промежуточными артефактами, исследование репозитория и последовательное уточнение результата.

3 Проектирование и реализация мультиагентной системы генерации документации

3.1 Назначение и требования к системе

Разрабатываемая система создаётся в составе проекта OSA (Open-Source Advisor), предназначенного для автоматизированного анализа и улучшения open-source репозиторий с использованием больших языковых моделей. Основная идея проекта заключается в том, чтобы принять репозиторий в качестве входных данных, проанализировать его структуру, метаданные и содержимое, а затем выполнить набор операций, направленных на повышение качества документационного сопровождения и общей готовности проекта к публикации и сопровождению.

В контексте настоящей работы центральным компонентом является механизм генерации и итеративного улучшения README-файла. В отличие от подхода, при котором весь документ формируется одним запросом к языковой модели, в OSA используется граф обработки, включающий сбор контекста, анализ намерения пользователя, планирование секций, генерацию отдельных частей, сборку итогового документа, самопроверку, точечное улучшение и запись результата. Такая организация позволяет ограничивать объём передаваемого контекста, использовать детерминированные правила для устойчивых секций и выполнять доработку результата на основе выявленных замечаний.

К функциональным требованиям системы относится возможность получения репозитория по URL, анализа его структуры и метаданных, выявления существующих документационных файлов, формирования контекста для языковой модели и генерации README.md. Кроме того, система должна поддерживать работу с пользовательским запросом, учитывать дополнительные материалы, например PDF-файл статьи, а также сохранять результат в целевой репозиторий. Отдельным функциональным

требованием является поддержка итеративного улучшения. После формирования первичного варианта README система должна оценивать результат, выявлять проблемные секции, определять необходимость повторной генерации или точечной правки и завершать процесс только после достижения критерия остановки либо после исчерпания заданного числа итераций. Такой подход позволяет рассматривать README не как статический текстовый вывод, а как артефакт, проходящий цикл проверки и уточнения.

К нефункциональным требованиям относятся модульность, расширяемость и воспроизводимость. Модульность необходима для независимого развития операций анализа, генерации, оценки и записи результата. Расширяемость важна из-за необходимости добавления новых секций README, новых типов документации, дополнительных провайдеров LLM и новых Git-платформ. Воспроизводимость обеспечивается за счёт конфигурационных файлов, явного задания параметров модели, ограничения токенов-бюджетов и фиксации сценариев запуска через CLI или Docker.

Система также должна учитывать ограничения, характерные для LLM-based решений. Передача всего репозитория в модель является неустойчивой из-за ограничений контекстного окна, поэтому требуется предварительный отбор релевантных данных. Ответы языковой модели должны иметь структурированный вид и проходить валидацию, поскольку произвольный текст не может использоваться как надёжный управляющий сигнал. Кроме того, результат генерации должен сохранять возможность последующей проверки разработчиком, так как даже при наличии самопроверки полностью исключить фактические ошибки невозможно.

Таким образом, назначение разрабатываемой системы состоит в автоматизации формирования и улучшения документации open-source репозитория, прежде всего README-файла, с использованием мультиагентного LLM-пайплайна. Система должна объединять анализ

репозитория, подготовку контекста, генерацию документа, оценку качества и итеративное уточнение результата в единую управляемую процедуру.

3.2 Общая архитектура программного комплекса

Программный комплекс OSA имеет модульную архитектуру, в которой обработка open-source репозитория представлена как последовательность этапов: получение пользовательского запроса, загрузка конфигурации, подготовка репозитория, выбор операций, выполнение изменений и сохранение результата.

В архитектуре OSA можно выделить два верхнеуровневых контура выполнения. Первый контур представляет собой классический CLI-сценарий: пользователь передаёт параметры запуска, система загружает конфигурацию, подготавливает репозиторий, формирует план операций и последовательно выполняет выбранные действия. Такой режим обеспечивает воспроизводимый запуск отдельных функций системы, например генерации README, docstring, requirements, workflow-файлов или отчётов.

Второй контур связан с агентным управлением процессом обработки репозитория. В этом режиме выполнение строится не только как последовательный запуск заранее выбранных модулей, а как workflow, в котором несколько специализированных агентов последовательно анализируют запрос, подготавливают репозиторий, выбирают операции, выполняют их, проверяют результат и завершают работу системы. Именно этот контур представляет наибольший интерес для настоящей работы, поскольку он отражает общий мультиагентный принцип организации OSA.

В рамках общей архитектуры пайплайн генерации README рассматривается как одна из операций OSA. Он использует общие инфраструктурные компоненты системы: конфигурационный слой, Git-слой, LLM-слой, механизм событий и запись результата. При этом внутренняя логика генерации README выделяется в отдельный специализированный

pipeline и рассматривается в следующих подразделах, поскольку именно она связана с задачей генерации и итеративного улучшения документации. Основные компоненты программного комплекса представлены в таблице 3.

Таблица 3 – Основные компоненты программного комплекса OSA

Компонент	Назначение
Пользовательский интерфейс	Получение параметров запуска, URL репозитория, пользовательского запроса и дополнительных материалов
Конфигурационный слой	Загрузка параметров Git, LLM, режима выполнения, токен-бюджетов и других настроек
Git-слой	Клонирование репозитория, получение метаданных, подготовка ветки, выполнение commit, push и pull request при необходимости
LLM-слой	Унифицированное взаимодействие с языковыми моделями, включая отправку запросов, обработку ответов, повторные попытки и fallback-модели
Реестр операций	Хранение доступных операций OSA и сведений об их назначении, аргументах и способе запуска
Операции OSA	Генерация README, community-файлов, docstring, requirements, CI/CD workflow, отчётов и других артефактов
Агентный контур управления	Анализ пользовательского запроса, построение плана, выполнение операций, проверка результата и повторное планирование при необходимости

Агентный контур управления OSA представлен на рисунке 2. Он показывает последовательность взаимодействия верхнеуровневых агентов и наличие обратной связи от этапа проверки к этапу планирования.

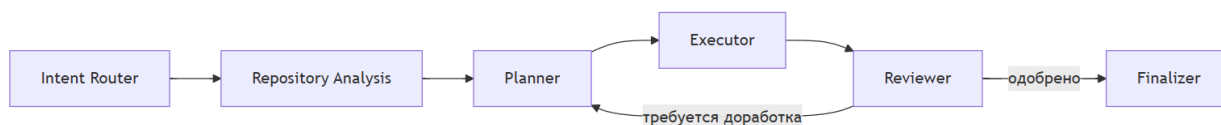


Рисунок 2 – Агентный контур управления OSA

Первым этапом агентного воркфлоу является определение намерения пользователя. Агент маршрутизации анализирует пользовательский запрос, дополнительный контекст и вложенные материалы, после чего определяет тип задачи и её область. Далее агент анализа репозитория подготавливает проект к обработке: получает доступ к репозиторию, собирает сведения о его структуре и метаданных, а также формирует данные, необходимые для планирования операций.

После анализа репозитория управление передаётся агенту планирования. Его задача состоит в выборе операций, которые должны быть выполнены для достижения цели пользователя. План может включать одну операцию, например генерацию README, или несколько действий, связанных с документацией, зависимостями, workflow-файлами и отчётами. Затем агент исполнения последовательно запускает выбранные операции и сохраняет полученные артефакты.

Отдельное значение имеет этап проверки. Reviewer-агент анализирует результат выполнения и получает подтверждение или обратную связь. Если результат принят, управление передаётся завершающему агенту, который формирует итоговое описание выполненных изменений и при необходимости инициирует публикацию результата через Git-механизмы. Если результат требует доработки, управление возвращается к агенту планирования, что позволяет скорректировать дальнейшие действия с учётом замечаний.

Более подробно агенты верхнеуровневого воркфлоу представлены в таблице 4.

Таблица 4 – Роли агентов верхнеуровневого воркфлоу OSA

Агент	Назначение	Результат работы
Intent Router	Определяет намерение пользователя и область задачи	Сформулированные цель и объем работы
Repository Analysis	Подготавливает репозиторий и собирает сведения о его структуре	Данные анализа репозитория и метаданные проекта
Planner	Выбирает операции, необходимые для выполнения запроса	План выполнения операций
Executor	Запускает операции из плана и собирает результаты	Артефакты выполнения и события операций
Reviewer	Проверяет результат и обрабатывает обратную связь	Решение о завершении или необходимости доработки
Finalizer	Завершает workflow и публикует изменения при необходимости	Итоговый результат, summary, commit, push или pull request

3.3 Архитектура пайплайна генерации README

Центральным компонентом разрабатываемой системы является пайплайн генерации README, встроенный в OSA как специализированная операция. Его назначение состоит в автоматическом создании или улучшении файла README.md на основе данных Git-репозитория, существующей документации, ключевых файлов проекта, пользовательского запроса и, при наличии, дополнительных материалов. В OSA генерация README

организована как граф обработки с явным состоянием и несколькими специализированными узлами.

Такая архитектура выбрана из-за неоднородности самого README-файла. В одном документе необходимо объединить описание назначения проекта, инструкции по установке, примеры использования, сведения о структуре проекта, ссылках на дополнительную документацию, лицензии и возможностях участия в разработке. Для Python-проектов это особенно важно, поскольку README может выступать не только страницей репозитория, но и описанием пакета на PyPI или Anaconda. Для Python-пакетов в состав README рекомендуется включать краткое описание, сведения об установке, quick-start пример, ссылки на документацию, community-раздел, информацию о лицензии и цитировании [18]. Кроме того, фрагменты кода являются значимыми элементами README, демонстрирующими использование библиотек и API [19]. Эти разделы опираются на разные источники информации: одни могут быть построены по метаданным репозитория, другие требуют анализа исходного кода, существующей документации или пользовательского запроса. Поэтому генерация документа как единого неструктурированного текста повышает риск пропуска важных сведений, нарушения логики разделов и появления фактических ошибок.

В OSA пайплайн README реализуется как граф, в котором каждый узел отвечает за отдельный этап обработки. В качестве входных данных используются URL репозитория, пользовательский запрос и дополнительный материал, если он был передан. Затем создаётся рабочий контекст операции, включающий конфигурацию, метаданные, prompt-шаблонов и средство взаимодействия с LLM. В процессе выполнения графа все промежуточные результаты сохраняются в общем состоянии: собранный контекст репозитория, определённое намерение пользователя, план секций, сгенерированные разделы, черновик README, результаты самопроверки, замечания и события выполнения операции. Для фиксации зависимостей операции и промежуточных результатов пайплайна используются структуры

ReadmeContext и ReadmeState; фрагменты их программной реализации приведены в приложениях А и Б.

После инициализации рабочего контекста запускается основная последовательность обработки, представленная на рисунке 3. Сначала пайплайн собирает сведения о репозитории: структуру файлов, существующий README, ключевые исходные файлы, примеры, тесты, документационные материалы и дополнительные вложения, если они были переданы пользователем. Полученные данные не передаются в LLM целиком, поскольку система учитывает ограничения контекстного окна и заранее отбирает только те элементы, которые могут быть полезны для генерации документа.

Далее выполняются анализ намерения пользователя и планирование структуры README. На этом этапе определяется режим работы пайплайна: создание документа с нуля, улучшение существующего README или частичное обновление отдельных разделов. После этого формируется план секций будущего документа. Важной особенностью является использование каталога допустимых секций, где для каждого раздела заданы название, приоритет, стратегия формирования и необходимые контекстные данные. За счёт этого структура README не создаётся языковой моделью произвольно, а формируется в рамках заранее определённой схемы.

После планирования секции генерируются отдельно и затем объединяются в единый Markdown-документ. Такой подход позволяет использовать разные стратегии формирования разделов: часть содержания создаётся с помощью LLM, а часть может быть получена детерминированно на основе метаданных, шаблонов и файлов репозитория. Затем собранный черновик проходит самопроверку. Если обнаружены недостатки, граф возвращает отдельные секции на повторную генерацию либо запускает точечную правку итогового README. При отсутствии существенных замечаний или после достижения максимального числа циклов улучшения результат передаётся на запись в репозиторий.

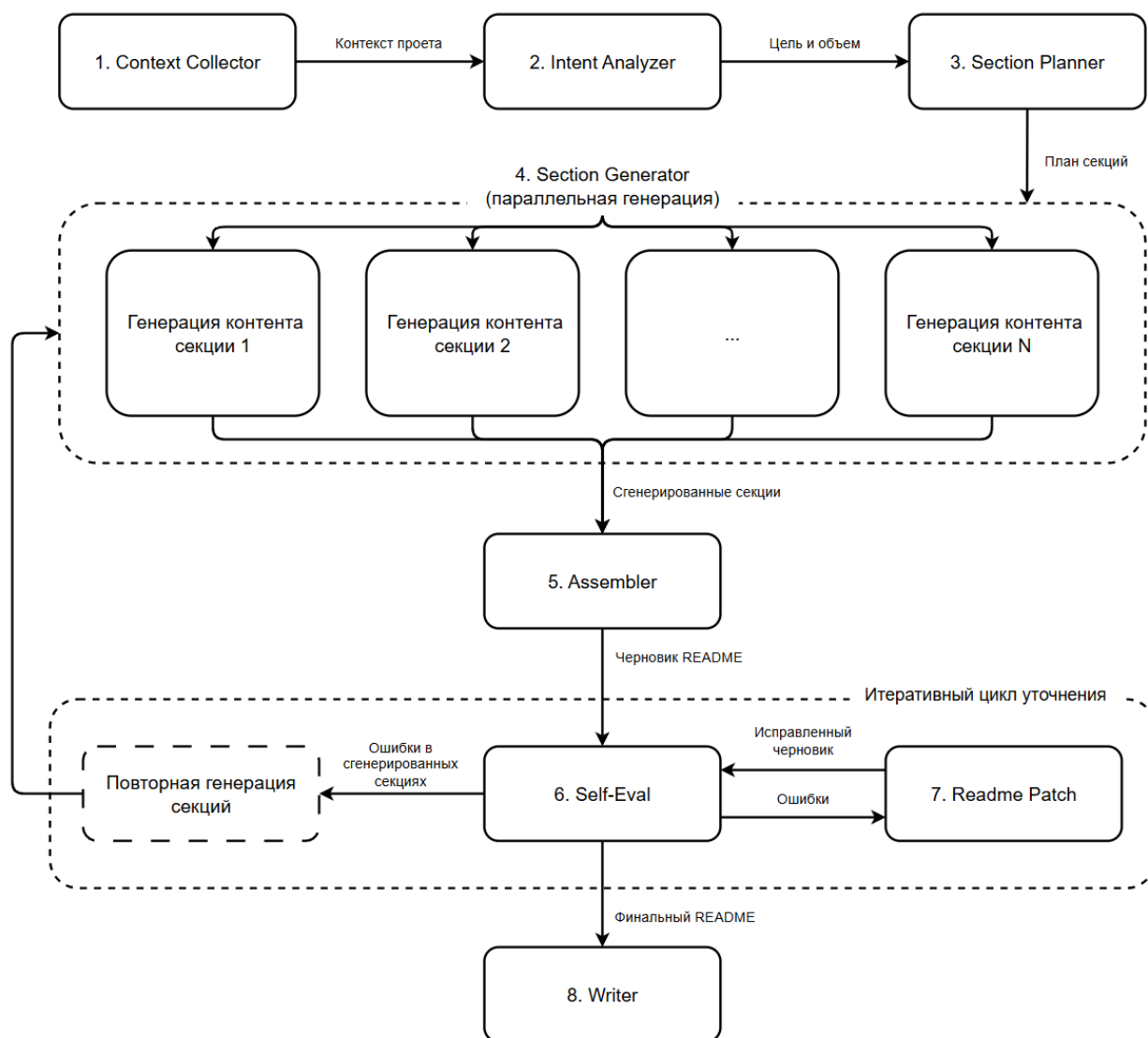


Рисунок 3 – Схема взаимодействия агентов пайплайна генерации README

Таким образом, архитектура пайплайна генерации README в OSA основана на графовой организации обработки, явном состоянии, каталоге секций и сочетании LLM-генерации с детерминированными правилами. Это позволяет рассматривать генерацию документа как управляемый процесс, включающий сбор контекста, планирование структуры, генерацию разделов, самопроверку и итеративное уточнение результата.

3.4 Агенты пайплайна генерации README и их функции

В предыдущем подразделе пайплайн генерации README был рассмотрен как граф обработки. Далее необходимо раскрыть функции его

отдельных узлов, поскольку именно распределение обязанностей между агентами обеспечивает управляемость процесса генерации. В рамках OSA каждый агент выполняет ограниченную подзадачу и возвращает не произвольный текст, а частичное обновление общего состояния графа: собранный контекст, результат анализа намерения, план секций, сгенерированный раздел, оценку качества или исправленный черновик. Такой подход позволяет рассматривать агентов как компоненты с явно заданными входами и выходами.

Использование общего состояния также важно для организации fan-out генерации секций. После построения плана каждая секция README может быть передана в отдельный вызов генератора, а результаты затем объединяются в общем состоянии пайплайна. Такая схема соответствует модели графовых воркфлоу, где узлы читают и обновляют состояние, а механизмы объединения результатов позволяют возвращать данные из параллельных или динамически созданных ветвей обработки [20].

В графе агентами являются узлы обработки, последовательно обновляющие общее состояние пайплайна. Их функции приведены в таблице 5.

Таблица 5 – Функции агентов пайплайна генерации README

Агент	Назначение	Результат работы	Роль LLM
Context Collector	Собирает и сокращает контекст репозитория: дерево файлов, существующий README, ключевые файлы, примеры, тесты и дополнительные материалы	Структурированный контекст репозитория	Используется для выбора ключевых файлов и анализа репозитория
Intent Analyzer	Определяет цель обработки README: создание с нуля, улучшение существующего	Описание намерения пользователя и области изменения	Используется в сложных сценариях; простые случаи могут

Агент	Назначение	Результат работы	Роль LLM
	документа или частичное обновление		обрабатываться правилами
Section Planner	Формирует план секций будущего README на основе контекста, намерения пользователя и каталога допустимых разделов	Упорядоченный список секций README	Используется для выбора релевантных секций
Section Generator	Генерирует содержимое отдельных секций README по сформированному плану	Сгенерированные разделы документа	Используется для LLM-секций; часть секций формируется детерминированно
Assembler	Объединяет сгенерированные секции в единый Markdown-документ или встраивает новые секции в существующий README	Черновик README	Используется только при слиянии с существующим документом
Self-Eval	Проверяет черновик README, выявляет недостатки и определяет необходимость доработки	Оценка качества, список замечаний и секции для повторной генерации	Используется для анализа качества результата
Readme Patch	Выполняет точечную доработку README по замечаниям самопроверки	Исправленный черновик README	Используется для редактирования итогового документа
Writer	Записывает итоговый README.md в репозиторий и фиксирует события выполнения	Финальный файл README.md и события операции	Не используется

Первым этапом пайплайна является работа агента **Context Collector**. Его задача состоит в подготовке исходного контекста, на основе которого будут работать последующие агенты. На этом этапе система получает дерево файлов репозитория, извлекает существующий README, определяет наличие тестов, примеров, документационных директорий и других вспомогательных

материалов. Если пользователь передал дополнительный файл, например PDF-статью, её содержимое также включается в рабочий контекст.

Принципиальная особенность данного этапа состоит в том, что репозиторий не передаётся в LLM целиком. Сначала система формирует представление структуры проекта, затем выделяет наиболее значимые файлы и считывает их содержимое с учётом token budget. Такой подход связан не только с техническим ограничением контекстного окна, но и с особенностями обработки длинного входа LLM. Модели с длинным контекстом не всегда устойчиво используют релевантную информацию: качество может снижаться, если важные сведения расположены внутри большого входного фрагмента, а не ближе к началу или концу [21]. Поэтому предварительный отбор контекста необходимым этапом повышения устойчивости генерации. Формирование контекста репозитория агентов изображено на рисунке 4.

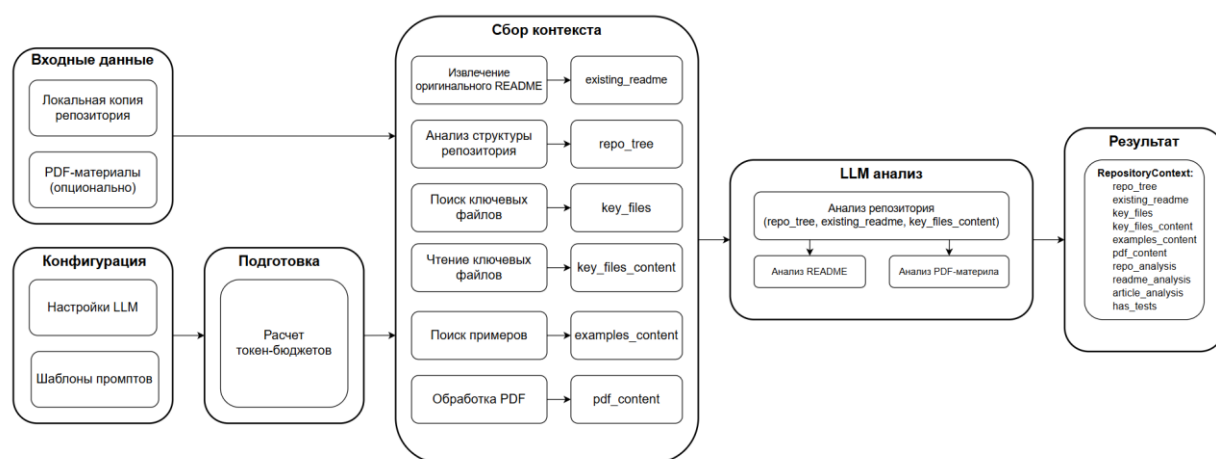


Рисунок 4 – Структурно-функциональная схема агента Context Collector

Логика работы Context Collector близка к идее retrieval-augmented generation (RAG), при которой перед генерацией выполняется извлечение релевантной информации из внешнего источника. В случае OSA таким источником выступает сам репозиторий: его дерево файлов, документация, конфигурационные файлы, примеры и ключевые исходные файлы. Подобный подход позволяет не полагаться только на параметрические знания языковой модели, а связывать генерацию с конкретными данными анализируемого проекта [22].

После подготовки контекста управление передаётся агенту **Intent Analyzer**. Он определяет, какую именно задачу необходимо решить: создать README с нуля, улучшить уже существующий документ или обновить только отдельные разделы. Такое разделение важно, поскольку разные режимы требуют разных стратегий обработки. Если README отсутствует, система должна сформировать документ полностью. Если README уже существует, пайплайн должен учитывать его текущее содержание и сохранять полезные фрагменты. Если пользователь просит изменить только конкретный раздел, вмешательство в остальную часть документа должно быть ограничено.

Intent Analyzer использует сведения о наличии README, пользовательский запрос, результаты анализа репозитория и информацию о дополнительных материалах. В простом случае, когда README отсутствует и пользователь не задаёт специальных требований, задача может быть определена без обращения к LLM. В более сложных сценариях применяется специализированный prompt-шаблон, результатом которого является структурированное описание намерения пользователя.

Следующим этапом является работа агента **Section Planner**. Его задача в построении структуры будущего документа. На основе контекста репозитория, намерения пользователя, анализа существующего README и каталога допустимых секций агент определяет, какие разделы должны быть включены в итоговый файл.

Важной особенностью является то, что структура README не создаётся языковой моделью произвольно. Система использует каталог секций, где заранее заданы внутренние имена разделов, заголовки, приоритеты, стратегии формирования и необходимые контекстные данные. LLM выбирает секции из ограниченного перечня, а система затем нормализует результат: удаляет дубли, устраняет пересечения и добавляет обязательные детерминированные разделы. Это снижает риск появления случайной или избыточной структуры документа.

После построения плана запускается **Section Generator**. Этот агент формирует содержимое отдельных секций README. В OSA используется fan-

out схема: каждая запланированная секция обрабатывается отдельным вызовом генератора, а результаты затем возвращаются в общее состояние графа. Такая организация соответствует map-reduce логике LangGraph, где Send API может использоваться для создания нескольких параллельных или динамически сформированных задач, а reducers применяются для объединения результатов в общем состоянии [23].

Для каждой секции используется её спецификация. В ней указано, какой контекст должен быть передан агенту: общий анализ репозитория, сведения о ключевых файлах, анализ существующего README, пользовательский запрос, данные из PDF или подсказки повторной генерации. Благодаря этому разные разделы получают разный набор данных. Например, секция Overview требует обобщения назначения проекта, секция Usage — примеров запуска, а секция Architecture — информации о структуре и компонентах кодовой базы.

Если пайплайн работает в режиме улучшения или обновления, Section Generator может учитывать уже существующую версию секции. Это позволяет не переписывать документ полностью, а сохранять корректные фрагменты исходного README. При повторной генерации после самопроверки агент также получает подсказки, сформированные на основе замечаний Self-Eval.

После генерации секций управление передаётся агенту **Assembler**. Он объединяет отдельные разделы в единый Markdown-документ. В режиме полной генерации Assembler упорядочивает секции по приоритету, добавляет заголовки и формирует целостный README. В режиме частичного обновления поведение отличается: новые или изменённые секции должны быть встроены в уже существующий документ без разрушения нецелевых частей. Поэтому Assembler занимает промежуточное положение между детерминированной сборкой и LLM-assisted редактированием: при полной генерации он может работать по правилам, а при слиянии с существующим README может использовать языковую модель.

На следующем этапе запускается агент **Self-Eval**. Он анализирует собранный черновик README с точки зрения полноты, ясности, структуры

Markdown, соответствия пользовательскому запросу и согласованности с данными о репозитории. Результат самопроверки имеет структурированный вид: агент возвращает оценку качества, список замечаний, перечень секций, требующих повторной генерации, и решение о возможности завершения пайплайна.

Структурированный формат ответа особенно важен для Self-Eval, поскольку его результат используется как управляющий сигнал для графа. В OSA агент возвращает данные в заданной структуре, что позволяет маршрутизировать выполнение: завершить пайплайн, регенерировать отдельные секции или запустить точечную правку.

Если Self-Eval обнаруживает проблему, локализованную в конкретной секции, соответствующий раздел может быть отправлен обратно в Section Generator. Если же проблема относится к документу в целом, запускается агент **Readme Patch**. Его задача состоит в точечной доработке уже собранного README: исправлении формулировок, устранении противоречий, улучшении связности, корректировке Markdown-структуры или добавлении недостающего пояснения.

Такой механизм позволяет не пересоздавать весь документ при каждой найденной проблеме. Это важно, поскольку полная повторная генерация может разрушить уже удачные фрагменты README. Readme Patch получает текущий черновик и замечания самопроверки, после чего формирует исправленную версию документа. Затем результат снова может быть передан на Self-Eval. Для предотвращения бесконечной самокоррекции в пайплайне используется ограничение на максимальное число циклов улучшения.

Последним агентом является **Writer**. Он не использует LLM и выполняет техническое завершение операции. Writer получает финальную версию README, приводит форматирование к ожидаемому виду, записывает файл README.md в целевой репозиторий и формирует события выполнения. С точки зрения архитектуры Writer является границей между внутренним состоянием пайплайна и реальным артефактом проекта. До этого момента

README существует как черновик в состоянии графа; после выполнения Writer он становится файлом репозитория и может быть использован в дальнейшем Git-сценарии OSA.

Таким образом, в рамках главы была рассмотрена проектно-реализационная основа разрабатываемой системы. Показано, что генерация README в OSA организована как управляемый граф обработки, включающий сбор контекста, анализ намерения пользователя, планирование структуры документа, генерацию секций, сборку, самопроверку, точечную доработку и запись результата. Распределение функций между агентами позволяет ограничить объём передаваемого в LLM контекста, использовать каталог секций, сочетать LLM-генерацию с детерминированными правилами и реализовать итеративное улучшение документации. Полученная архитектура служит основой для дальнейшей экспериментальной оценки качества сгенерированных README-файлов.

4.1 Цель и методика экспериментального исследования

Целью экспериментального исследования является оценка эффективности разработанной системы генерации и итеративного улучшения README-файлов для open-source репозиториях.

Методика эксперимента строится на сравнении разработанного подхода с альтернативными способами автоматической генерации README. В качестве базового подхода рассматривается прямая генерация README с помощью LLM, при которой модель получает входной контекст и формирует документ. Дополнительно используется внешнее open-source решение ReadmeAI [24], предназначенное для автоматической генерации README-файлов. Такой выбор позволяет сопоставить OSA как с простым LLM-based baseline, так и со специализированным инструментом, решающим близкую задачу [25].

Эксперимент проводится на наборе открытых репозиториях, разделённых на две группы: репозитории общего назначения и научно-образовательные репозитории. Такое разделение важно, поскольку OSA ориентирована прежде всего на повышение качества научных open-source проектов, где документация должна не только описывать программный интерфейс, но и помогать воспроизвести исследовательский результат.

Для проверки устойчивости подхода эксперимент проводится с использованием нескольких языковых моделей. В статьях по OSA рассматриваются GPT-4.1, Claude 4 Sonnet и Gemma3 27B. Такое сравнение позволяет оценить не только качество итогового README, но и зависимость системы от выбранной модели. Для мультиагентной системы это особенно важно: если архитектура действительно ограничивает контекст, структурирует задачу и использует самопроверку, итоговое качество должно

быть менее чувствительным к замене модели, чем при прямой генерации одним LLM-запросом.

Оценка качества README выполняется с использованием LLM-as-a-judge подхода на основе G-Eval. Данный метод применяет языковую модель для оценки сгенерированного текста по заданным критериям и был предложен как способ автоматической оценки NLG-результатов, в том числе в задачах, где использование простых reference-based метрик недостаточно отражает качество текста [26]. В рамках эксперимента используется кастомный prompt, оценивающий соответствие README структуре репозитория, ясность описания назначения проекта, наличие инструкций установки и запуска, наличие примеров использования, корректность сведений о зависимостях, а также общую читаемость и структурированность документа [25].

При интерпретации результатов необходимо учитывать, что LLM-based оценка не является полной заменой экспертной проверки. README-файл содержит как текстовую, так и фактическую составляющую: документ должен быть понятным, но при этом не должен содержать утверждений, не подтверждённых содержимым репозитория. Поэтому результаты автоматической оценки рассматриваются как сравнительная метрика качества, позволяющая сопоставить OSA, прямую генерацию через LLM и ReadmeAI на одинаковом наборе репозиториях. Итоговый анализ должен учитывать как численные значения метрики, так и содержательные различия между исходными и сгенерированными README-файлами.

Таким образом, экспериментальная методика направлена на проверку того, обеспечивает ли мультиагентный пайплайн OSA более качественную генерацию README по сравнению с однократным LLM-вызовом и существующим специализированным инструментом. В следующих подразделах рассматриваются состав экспериментального набора, критерии оценки и полученные результаты.

4.2 Экспериментальный набор репозитория

Экспериментальное исследование проводилось на наборе открытых GitHub-репозиториях, разделенных на две группы: проекты общего назначения и научно-образовательные проекты. Такое разделение важно для настоящей работы, поскольку разрабатываемая система ориентирована не только на типовые open-source библиотеки, но и на репозитории, связанные с исследовательским кодом, учебными материалами, моделями машинного обучения и инструментами анализа данных.

Выбор экспериментального набора обусловлен необходимостью проверить работу системы на репозиториях с различной структурой и разным уровнем документационного сопровождения. В набор входят библиотеки, CLI-инструменты, образовательные материалы, репозитории с моделями машинного обучения, проекты для обработки изображений, API-инструменты и исследовательские разработки. Такая неоднородность позволяет оценить, насколько пайплайн генерации README способен работать не только с простыми проектами, но и с репозиториями, в которых присутствуют несколько директорий, конфигурационные файлы, примеры, зависимости и существующая документация.

В качестве основного критерия отбора использовалась открытость репозитория и возможность автоматизированного анализа его структуры. Для задачи генерации README особенно важно, чтобы система могла получить дерево файлов, существующий README, сведения о зависимостях, примеры использования и ключевые файлы проекта. Полный перечень репозиториях представлен в приложении В.

Таким образом, экспериментальный набор позволяет проверить систему в условиях, близких к реальному использованию: на открытых репозиториях разного назначения, структуры и качества исходной документации.

4.3 Критерии оценки качества README-файлов

Оценка качества README-файлов выполнялась с использованием LLM-as-a-judge подхода на основе G-Eval. Данный выбор связан с тем, что README является текстовым артефактом со сложной структурой: для него недостаточно проверить только сходство с эталонным текстом, поскольку качественный README может отличаться от исходного документа по структуре, но при этом лучше описывать назначение проекта, установку, запуск и примеры использования. G-Eval рассматривается как метод автоматической оценки NLG-результатов с использованием LLM, chain-of-thought и form-filling подхода; авторы метода также отмечают, что традиционные reference-based метрики, такие как BLEU и ROUGE, часто слабо отражают качество открытых текстовых ответов [26].

В эксперименте сравнивались README исходный и сгенерированный системой. В качестве EXPECTED_OUTPUT использовался исходный документ репозитория, а в качестве ACTUAL_OUTPUT — сгенерированный README вместе со структурой репозитория в формате JSON. Такое представление позволяет оценщику проверять не только читаемость текста, но и согласованность сгенерированной документации с фактической структурой проекта.

Критерии оценки были сформулированы так, чтобы покрыть основные свойства README как документационной точки входа в open-source репозиторий. Они включают соответствие структуре репозитория, ясность описания назначения проекта, наличие инструкций установки и настройки, наличие примеров использования, корректность сведений о зависимостях и общую читаемость документа. Критерии представлены в таблице 6.

Таблица 6 – Критерии оценки качества README-файлов

№	Критерий оценки	Проверяемый аспект
1	Соответствие структуре репозитория	Сгенерированный README не должен противоречить дереву файлов и фактическому содержимому проекта
2	Описание назначения проекта	Документ должен ясно и корректно объяснять цель репозитория и его основную функциональность
3	Инструкции установки и настройки	README должен содержать понятные шаги установки, подготовки окружения или запуска проекта
4	Примеры использования	Документ должен включать примеры, демонстрирующие основные сценарии применения проекта
5	Зависимости и требования	README должен корректно отражать зависимости, requirements или условия запуска
6	Читаемость и структура	Документ должен быть логично организован, хорошо читаем и не содержать сбивающих формулировок

Отдельно учитывалось, что структура сгенерированного README не обязана полностью совпадать со структурой исходного документа. Важнее, чтобы итоговый файл проходил содержательную проверку по перечисленным критериям. Это позволяет корректно оценивать случаи, когда исходный README был неполным или плохо структурированным, а система сформировала более полезный документ с другой организацией разделов.

Prompt-шаблон, использованный для автоматической оценки качества README-файлов, приведён в приложении Г.

Таким образом, выбранная схема оценки позволяет сравнивать README-файлы не только по формальному сходству с исходным документом,

но и по их практической полезности для пользователя репозитория. Использование структуры репозитория в составе входных данных снижает риск оценки сгенерированного текста вне контекста проекта, а набор критериев позволяет учитывать как содержательную полноту, так и читаемость итогового документа.

4.4 Анализ результатов экспериментального исследования

Результаты экспериментального сравнения приведены в таблице 7. В качестве сравниваемых подходов рассматривались базовый вариант прямой генерации README с помощью LLM, специализированные инструменты ReadmeAI и LARCH, а также разработанный подход OSA. Оценка проводилась отдельно для репозиториях общего назначения, научных репозиториях и всего экспериментального набора. Значение метрики находится в диапазоне от 0 до 1, где большее значение соответствует более высокому качеству README-файла.

Таблица 7 – Результаты оценки качества README-файлов

Тип репозитория	GPT 4.1				Claude 4 Sonnet			Gemma3 27b		
	Base	RDAI	Larch	OSA	Base	Larch	OSA	Base	Larch	OSA
Общий	0.31	0.72	0.76	0.90	0.17	0.71	0.87	0.35	0.64	0.85
Научный	0.40	0.77	0.81	0.89	0.33	0.77	0.92	0.25	0.69	0.89
Все	0.36	0.75	0.77	0.89	0.25	0.73	0.89	0.30	0.63	0.87

Полученные результаты показывают, что OSA демонстрирует наилучшее качество генерации README во всех рассматриваемых группах репозиториях и при использовании всех выбранных языковых моделей.

Отдельно следует отметить устойчивость результатов на разных типах репозиториях. Различия между общими и научными проектами остаются

небольшими, что указывает на применимость пайплайна не только к типовым open-source библиотекам, но и к репозиториям научно-образовательного характера. Это важно для OSA, поскольку система ориентирована на улучшение качества документации именно в открытых проектах, где структура, полнота и воспроизводимость README имеют практическое значение.

Таким образом, результаты эксперимента подтверждают практическую пользу мультиагентного пайплайна генерации README. Разработанный подход сохраняет высокое качество при изменении типа репозитория и используемой LLM, а также демонстрирует преимущество над baseline-подходом и специализированными аналогами.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была разработана мультиагентная система генерации и итеративного улучшения README-файлов для репозитория с открытым исходным кодом. В работе был проведён анализ предметной области и существующих решений, рассмотрены современные подходы к применению LLM в задачах программной инженерии, а также спроектирована архитектура пайплайна, включающего сбор контекста, анализ намерения пользователя, планирование секций, генерацию, самопроверку и доработку итогового документа.

Разработанный модуль был реализован в составе проекта OSA. Экспериментальное сравнение с baseline-подходом и существующими инструментами показало устойчивое преимущество OSA при разных типах репозитория и используемых языковых моделях. Полученные результаты подтверждают, что мультиагентная организация процесса и механизм итеративного улучшения позволяют повысить качество автоматически генерируемой документации и могут быть использованы для автоматизации сопровождения open-source проектов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. GitHub Docs. GitHub Best practices for repositories [Электронный ресурс]. – URL: <https://docs.github.com/en/repositories/creating-and-managing-repositories/best-practices-for-repositories> (дата обращения: 03.05.2026).
2. Luo Q. et al. Repoagent: An llm-powered open-source framework for repository-level code documentation generation //Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. – 2024. – С. 436-464.
3. Yang D. et al. Docagent: A multi-agent system for automated code documentation generation //Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations). – 2025. – С. 460-471.
4. OSA: репозиторий проекта / AIM Club // GitHub [Электронный ресурс]. – URL: <https://github.com/aimclub/OSA> (дата обращения: 03.05.2026).
5. Venigalla A. S. M., Chimalakonda S. An empirical study on correlation between readme content and project popularity //arXiv preprint arXiv:2206.10772. – 2022.
6. Mujahid S., Abdalkareem R., Shihab E. What are the characteristics of highly-selected packages? A case study on the npm ecosystem //Journal of Systems and Software. – 2023. – Т. 198. – С. 111588.
7. Prana G. A. A. et al. Categorizing the content of github readme files //Empirical Software Engineering. – 2019. – Т. 24. – №. 3. – С. 1296-1327.
8. GitHub Docs. Basic writing and formatting syntax [Электронный ресурс]. – URL: <https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax> (дата обращения: 03.05.2026).

9. Gaughan M. et al. The introduction of README and CONTRIBUTING files in open source software development //2025 IEEE/ACM 18th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE). – IEEE, 2025. – C. 191-202.
10. Treude C., Middleton J., Atapattu T. Beyond accuracy: Assessing software documentation quality //Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. – 2020. – C. 1509-1512.
11. Koreeda Y. et al. Larch: Large language model-based automatic readme creation with heuristics //Proceedings of the 32nd ACM International Conference on Information and Knowledge Management. – 2023. – C. 5066-5070.
12. Wang H. et al. Repomaster: Autonomous exploration and understanding of github repositories for complex task solving //arXiv preprint arXiv:2505.21577. – 2025.
13. Hou X. et al. Large language models for software engineering: A systematic literature review //ACM Transactions on Software Engineering and Methodology. – 2024. – T. 33. – №. 8. – C. 1-79.
14. He J., Treude C., Lo D. Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead //ACM Transactions on Software Engineering and Methodology. – 2025. – T. 34. – №. 5. – C. 1-30.
15. Hong S. et al. MetaGPT: Meta programming for a multi-agent collaborative framework //The twelfth international conference on learning representations. – 2023.
16. Qian C. et al. Chatdev: Communicative agents for software development //Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers). – 2024. – C. 15174-15186.

17. Nguyen D. S. H. et al. Teamwork makes the dream work: LLMs-Based Agents for GitHub README. MD Summarization //Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering. – 2025. – С. 621-625.
18. pyOpenSci. README File Guidelines and Resources [Электронный ресурс]. – URL: <https://www.pyopensci.org/python-package-guide/documentation/repository-files/readme-file-best-practices.html> (дата обращения: 04.05.2026).
19. Sitthithanasakul S. et al. Do Developers Present Proficient Code Snippets in Their README Files? An Analysis of PyPI Libraries in GitHub //Journal of Information Processing. – 2023. – Т. 31. – С. 679-688.
20. LangChain. LangGraph Graph API [Электронный ресурс]. – URL: <https://docs.langchain.com/oss/python/langgraph/graph-api> (дата обращения: 05.05.2026).
21. Liu N. F. et al. Lost in the middle: How language models use long contexts //Transactions of the association for computational linguistics. – 2024. – Т. 12. – С. 157-173.
22. Gao Y. et al. Retrieval-augmented generation for large language models: A survey //arXiv preprint arXiv:2312.10997. – 2023. – Т. 2. – №. 1. – С. 32.
23. OpenAI. Introducing Structured Outputs in the API [Электронный ресурс]. – URL: <https://openai.com/index/introducing-structured-outputs-in-the-api/> (дата обращения: 05.05.2026).
24. Readme-AI. Генерация файлов README с использованием искусственного интеллекта [Электронный ресурс]. – URL: <https://github.com/eli64s/readme-ai> (дата обращения: 07.05.2026).
25. Nikitin N. et al. An LLM-Powered Tool for Enhancing Scientific Open-Source Repositories //Championing Open-source DEvelopment in ML Workshop@ ICML25.

26. Liu Y. et al. G-eval: NLG evaluation using gpt-4 with better human alignment //Proceedings of the 2023 conference on empirical methods in natural language processing. – 2023. – C. 2511-2522.

Приложение А. Программная реализация структуры ReadmeContext

```
"""Shared dependency-injection container passed to every README generation node."""
```

```
from osa_tool.config.settings import ConfigManager
from osa_tool.core.git.metadata import RepositoryMetadata
from osa_tool.core.llm.llm import ModelHandler,
ModelHandlerFactory
```

```
class ReadmeContext:
```

```
"""Shared dependencies for all README generation nodes."""
```

```
    def __init__(self, config_manager: ConfigManager, metadata:
RepositoryMetadata) -> None:
        self.config_manager = config_manager
        self.metadata = metadata
        self.prompts = config_manager.get_prompts()
        self.model_handler: ModelHandler =
ModelHandlerFactory.build(config_manager.get_model_settings("re
adme"))
```

Приложение Б. Программная реализация структуры ReadmeState

```
"""Mutable workflow state for the README generation LangGraph
pipeline."""

from typing import Annotated, Literal

from pydantic import BaseModel, ConfigDict, Field

from osa_tool.core.models.event import OperationEvent
from
osa_tool.operations.docs.readme_generation.pipeline.llm_schemas
import SelfEvalIssue
from osa_tool.operations.docs.readme_generation.pipeline.models
import (
    RepositoryContext,
    SectionResult,
    SectionSpec,
    TaskIntent,
)

AssemblyMode = Literal["full_compose", "merge_existing"]

def _merge_dicts(left: dict, right: dict) -> dict:
    """Reducer: merge two dicts (right overwrites overlapping
keys)."""
    merged = left.copy()
    merged.update(right)
    return merged

class ReadmeState(BaseModel):
    """Full mutable state flowing through the README generation
graph."""

    model_config = ConfigDict(arbitrary_types_allowed=True)

    # Inputs
    repo_url: str
    attachment: str | None = None
    user_request: str | None = None

    # Collected context
    context: RepositoryContext | None = None

    # Intent analysis
    intent: TaskIntent | None = None

    # Section plan
    section_plan: list[SectionSpec] =
```

```

Field(default_factory=list)

    # Generated sections (merge reducer for parallel Send
writes)
    sections: Annotated[dict[str, SectionResult], _merge_dicts]
= Field(default_factory=dict)
    section_errors: Annotated[dict[str, str], _merge_dicts] =
Field(default_factory=dict)

    # Current section (set per-Send by fan-out)
current_section: SectionSpec | None = None

    # Assembly & self-eval refinement
readme_draft: str | None = None
refinement_score: float | None = None
refinement_structured_issues: list[SelfEvalIssue] =
Field(default_factory=list)
refinement_effective_finish: bool = False
refinement_issues: list[str] = Field(default_factory=list)
refinement_cycles: int = 0
max_refinement_cycles: int = 3
sections_to_rerun: list[str] = Field(default_factory=list)
section_regeneration_hints: dict[str, str] =
Field(default_factory=dict)

    # Output
events: list[OperationEvent] = Field(default_factory=list)

```


Приложение В. Экспериментальный набор репозиторий

<https://github.com/AntonOsika/gpt-engineer>,
<https://github.com/THUDM/ChatGLM-6B>,
<https://github.com/OpenEthan/SMSBoom>,
<https://github.com/lra/mackup>, <https://github.com/chenfei-wu/TaskMatrix>,
<https://github.com/hacksider/Deep-Live-Cam>,
<https://github.com/google/python-fire>,
<https://github.com/stitionai/devika>,
<https://github.com/Pythagora-io/gpt-pilot>,
<https://github.com/CorentinJ/Real-Time-Voice-Cloning>,
<https://github.com/encode/httpx>,
<https://github.com/lss233/kirara-ai>,
<https://github.com/assafelovic/gpt-researcher>,
<https://github.com/mkdocs/mkdocs>,
<https://github.com/ageitgey/facerecognition>,
<https://github.com/donnemartin/system-design-primer>,
<https://github.com/chatanywhere/GPTAPIfree>,
<https://github.com/Asabeneh/30-Days-Of-Python>,
<https://github.com/kaixindelele/ChatPaper>,
<https://github.com/twintproject/twint>,
<https://github.com/pallets/flask>,
<https://github.com/charlax/professional-programming>,
<https://github.com/ethereum/EIPs>, <https://github.com/xai-org/grok-1>,
<https://github.com/eriklindernoren/PyTorch-GAN>,
<https://github.com/public-apis/public-apis>,
<https://github.com/Z4nzu/hackintool>,
<https://github.com/gto76/python-cheatsheet>,
<https://github.com/danielgatis/rembg>, <https://github.com/bregman-arie/devops-exercises>,
<https://github.com/Vision-CAIR/MiniGPT-4>,
<https://github.com/Jack-Cherish/python-spider>,
<https://github.com/openai/chatgpt-retrieval-plugin>,
<https://github.com/black-forest-labs/flux>, <https://github.com/sb-ai-lab/EmotiEffLib>,
<https://github.com/sb-ai-lab/Ride>,
<https://github.com/hse-cs/delPezzo>,
<https://github.com/deeppavlov/chatsky>,
<https://github.com/deeppavlov/dialog2graph>,
<https://github.com/corl-team/verl-loras>, <https://github.com/sb-ai-lab/Eco2AI>,
https://github.com/AIRI-Institute/GENA_LM,
<https://github.com/AIRI-Institute/eco4cast>,
<https://github.com/Vishnu-tppr/Camouflage-AI>,
<https://github.com/tbhvishal/Python-Weather-Info-App>,
<https://github.com/readytensor/rt-repo-assessment>,
<https://github.com/DrpidamenteNanjesha/TagGenerator>,
<https://github.com/stephenombuya/Code-Contribution-Analyzer>

Приложение Г. Prompt-шаблон для оценки качества README

Determine whether the AI-generated Readme file (ACTUAL_OUTPUT) is better than the original one (EXPECTED_OUTPUT).

ACTUAL_OUTPUT contains two fields: readme (generated README) and repo_structure (JSON tree).

Generated README content must be consistent with the provided repository structure.

Your goal is to determine which text is better using the evaluation steps.

"Step 1: Does the provided structure of the repository address README content?",

"Step 2: Does the README provide a clear and accurate overview of the repository purpose?",

"Step 3: Are installation and setup instructions included and easy to follow?",

"Step 4: Are usage examples provided and do they clearly demonstrate functionality?",

"Step 5: Are dependencies or requirements listed appropriately?",

"Step 6: Is the README easy to read, well-structured, and free of confusing language?"